1. Test Codes

a) I2C check

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  while (!Serial);
  Serial.println("\nI2C Scanner");
}

void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127; address++ ) {
    // The i2c_scanner uses the return value of
    // the Write.endTransmisstion to see if
    // a device did acknowledge to the address.
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
```

```
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address < 16) Serial.print("0");
      Serial.print(address, HEX);
      Serial.println("  !");
      nDevices++;
    }
  }
  if (nDevices == 0)
    Serial.println("No I2C devices found\n");
  else
    Serial.println("done\n");

  delay(5000);           // wait 5 seconds for next scan
}
```

b) Motor Test

```
/*
 * ChromoBot - Motor Test
 * ----------------------
 * 1. Forward (2s)
 * 2. Backward (2s)
 * 3. Spin Left (2s)
 * 4. Spin Right (2s)
 * 5. Stop (2s)
 */

// --- MOTOR PINS ---
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;

void setup() {
  // Set all motor pins as outputs
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);
  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);

  Serial.begin(9600);
  Serial.println("--- MOTOR TEST STARTING ---");
  delay(2000);
}

void loop() {
```

```
  // 1. FORWARD
  Serial.println("FORWARD");
  move(100, 100);
  delay(2000);
  stopMotors(); delay(1000);

  // 2. BACKWARD
  Serial.println("BACKWARD");
  move(-100, -100);
  delay(2000);
  stopMotors(); delay(1000);

  // 3. SPIN LEFT
  Serial.println("SPIN LEFT");
  move(-100, 100);
  delay(2000);
  stopMotors(); delay(1000);

  // 4. SPIN RIGHT
  Serial.println("SPIN RIGHT");
  move(100, -100);
  delay(2000);
  stopMotors(); delay(2000);
}

// --- HELPER FUNCTION ---
```

```
void move(int l, int r) {
  // Left Motor Logic
  if(l > 0) {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, l);
  } else {
    digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, abs(l));
  }

  // Right Motor Logic
  if(r > 0) {
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, r);
  } else {
    digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, abs(r));
  }
}

void stopMotors() {
  analogWrite(L_PWM, 0); analogWrite(R_PWM, 0);
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}
```

c) Ultrasonic sensor

```
/*
 * ChromoBot - 3 Ultrasonic Sensor Test
 * ---------------------------------------------
 * Upload this to verify all sensors are working.
 * Open Serial Monitor at 9600 baud.
 */

// --- PINS ---
const int TRIG_L = 7;   const int ECHO_L = 8;
const int TRIG_MID = 10; const int ECHO_MID = 9;
const int TRIG_R = 11;  const int ECHO_R = 12;

void setup() {
  Serial.begin(9600);

  pinMode(TRIG_L, OUTPUT); pinMode(ECHO_L, INPUT);
  pinMode(TRIG_MID, OUTPUT); pinMode(ECHO_MID, INPUT);
  pinMode(TRIG_R, OUTPUT); pinMode(ECHO_R, INPUT);

  Serial.println("--- ULTRASONIC TEST STARTED ---");
  Serial.println("L = Left | M = Middle | R = Right");
  delay(1000);
}

void loop() {
```

```
  // Read all three
  long distL = readDistance(TRIG_L, ECHO_L);
  delay(10); // Small pause to prevent interference
  long distM = readDistance(TRIG_MID, ECHO_MID);
  delay(10);
  long distR = readDistance(TRIG_R, ECHO_R);

  // Print nicely
  Serial.print("L: ");
  Serial.print(distL);
  Serial.print(" cm  |  M: ");
  Serial.print(distM);
  Serial.print(" cm  |  R: ");
  Serial.print(distR);
  Serial.println(" cm");

  delay(500); // Update every half second
}

// Helper function
long readDistance(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long duration = pulseIn(echo, HIGH, 30000); // 30ms timeout (~5m)
```

```
  if (duration == 0) return 999; // Return 999 if no echo (out of range/disconnected)
  return duration * 0.0343 / 2;
}
```

d) IR array truth test

```
/*
 * IR Sensor Truth Test
 * --------------------
 * READ THIS OUTPUT TO KNOW YOUR LOGIC:
 * - Put Sensor on WHITE -> Note the number (0 or 1).
 * - Put Sensor on BLACK -> Note the number (0 or 1).
 */

// PINS (Based on your previous code)
const int S1 = 44;
const int S2 = 45;
const int S3 = 46;
const int S4 = 47;
const int S5 = 48;

void setup() {
  pinMode(S1, INPUT);
  pinMode(S2, INPUT);
  pinMode(S3, INPUT);
  pinMode(S4, INPUT);
  pinMode(S5, INPUT);

  Serial.begin(9600);
  Serial.println("--- IR SENSOR TEST STARTED ---");
  delay(1000);
```

```
}

void loop() {
  // Read all sensors
  int v1 = digitalRead(S1);
  int v2 = digitalRead(S2);
  int v3 = digitalRead(S3);
  int v4 = digitalRead(S4);
  int v5 = digitalRead(S5);

  // Print to Serial Monitor
  Serial.print("S1:"); Serial.print(v1); Serial.print("  ");
  Serial.print("S2:"); Serial.print(v2); Serial.print("  ");
  Serial.print("S3:"); Serial.print(v3); Serial.print("  ");
  Serial.print("S4:"); Serial.print(v4); Serial.print("  ");
  Serial.print("S5:"); Serial.println(v5);

  delay(200); // Slow down so you can read it
}
```

2.Line following working code

```
/*
 * ChromoBot - Final Logic (Longer Alignment Fix)
 * ----------------------------------------------
 * SPEEDS: Straight = 60 | Rotate = 100
 * ALIGN TIME: Increased to 1200ms for 26cm robot length.
 * * 1. BEND: Stop 1s -> Fwd 60 (1.2s) -> Stop 1s -> Rotate -> Wait for Mid 3.
 * 2. DRIFT: Stop 1s -> Rotate [Limit] -> Wait for Mid 3.
 * 3. NORMAL: PID Speed 60.
 */

// --- PIN DEFINITIONS ---
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;
const int S1 = 44; const int S2 = 45; const int S3 = 46; const int S4 = 47; const int S5 = 48;

// --- TUNING (UPDATED) ---
#define PID_SPEED   60
#define ALIGN_SPEED 60
#define TURN_SPEED  100

#define DRIFT_LIMIT 400     // Max time to rotate for drift (ms)

// INCREASED THIS TO 1200 (1.2 Seconds)
#define ALIGN_TIME  1200
```

```
// --- LOGIC MAP ---
#define LINE 1   // White
#define FLOOR 0  // Black

int lastError = 0;
bool isRunning = true;

#define KP 18.0
#define KD 22.0

void setup() {
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);
  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);
  pinMode(S1, INPUT); pinMode(S2, INPUT); pinMode(S3, INPUT); pinMode(S4, INPUT);
pinMode(S5, INPUT);

  Serial.begin(9600);
  Serial.println("--- READY: 1.2s Align Time ---");
  delay(2000);
}

void loop() {
  if (!isRunning) return;

  int s1 = digitalRead(S1);
  int s2 = digitalRead(S2);
```

```
int s3 = digitalRead(S3);
int s4 = digitalRead(S4);
int s5 = digitalRead(S5);
int activeSensors = s1 + s2 + s3 + s4 + s5;

// CHECK FOR LINE LOSS (Pink Section)
if (activeSensors == 0) {
  delay(100); // 100ms Debounce
  if (digitalRead(S3) == FLOOR) {
      stopMotors();
      isRunning = false;
      return;
  }
}

// ==========================================
// PRIORITY 1: BEND DETECTION (4 or 5 Sensors)
// ==========================================
if (activeSensors >= 4) {
  if (s1 == LINE && s2 == LINE && s3 == LINE && s4 == LINE) {
    handleBend(-1); // Left Bend
    return;
  }
  if (s2 == LINE && s3 == LINE && s4 == LINE && s5 == LINE) {
    handleBend(1);  // Right Bend
    return;
```

```
  }
}

// ==========================================
// PRIORITY 2: DRIFT DETECTION (3 Sensors)
// ==========================================
if (activeSensors == 3) {
  if (s1 == LINE && s2 == LINE && s3 == LINE) {
    handleDrift(-1); // Left Drift
    return;
  }
  if (s3 == LINE && s4 == LINE && s5 == LINE) {
    handleDrift(1); // Right Drift
    return;
  }
}

// ==========================================
// PRIORITY 3: NORMAL PID
// ==========================================
if (activeSensors > 0) {
  int error = (s1 * -4) + (s2 * -2) + (s3 * 0) + (s4 * 2) + (s5 * 4);
  error = error / activeSensors;

  int P = error;
  int D = error - lastError;
```

```
    int correction = (KP * P) + (KD * D);
    lastError = error;

    move(PID_SPEED + correction, PID_SPEED - correction);
  }
}

// --- MANEUVERS ---

void handleBend(int direction) {
  // 1. Stop 1s
  stopMotors(); delay(1000);

  // 2. Blind Forward (1200ms now)
  // Main loop is paused here, so it ignores black floor.
  move(ALIGN_SPEED, ALIGN_SPEED);
  delay(ALIGN_TIME);

  // 3. Stop 1s
  stopMotors(); delay(1000);

  // 4. Rotate until Center Found
  spinUntilMiddle3(direction, 6000);
}

void handleDrift(int direction) {
```

```
  stopMotors(); delay(1000);
  spinUntilMiddle3(direction, DRIFT_LIMIT);
}

void spinUntilMiddle3(int direction, int timeoutMs) {
  unsigned long startTime = millis();

  // SPIN LOGIC (Speed 100)
  if (direction == -1) {
    // Spin Left (Left Back, Right Fwd)
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, TURN_SPEED);
    digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, TURN_SPEED);
  } else {
    // Spin Right
    digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, TURN_SPEED);
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, TURN_SPEED);
  }

  delay(100); // Clear state

  while (millis() - startTime < timeoutMs) {
    int s2 = digitalRead(S2);
    int s3 = digitalRead(S3);
    int s4 = digitalRead(S4);

    // Target: Middle 3 White
```

```
    if (s2 == LINE && s3 == LINE && s4 == LINE) {
      stopMotors();
      delay(200);
      return;
    }
  }
  stopMotors();
}

// --- MOTORS ---
void move(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);

  if (leftSpeed >= 0) {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, leftSpeed);
  } else {
    digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, abs(leftSpeed));
  }

  if (rightSpeed >= 0) {
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, rightSpeed);
  } else {
    digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, abs(rightSpeed));
  }
}
```

```
void stopMotors() {
  analogWrite(L_PWM, 0); analogWrite(R_PWM, 0);
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}
```

3.Tunnel code

```
/*
 * ChromoBot - Tunnel Only (Strict AND Logic)
 * ---------------------------------------------
 * PINS: Left(7,8) | Right(11,12)
 * RANGE: 6cm - 20cm
 * BEHAVIOR:
 * - Robot moves ONLY if BOTH walls are detected.
 * - If one wall is missing/far (>20cm), Robot STOPS.
 * - If one wall is too close (<6cm), Robot STOPS.
 */

// --- MOTOR PINS ---
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;

// --- ULTRASONIC PINS (Corrected) ---
const int TRIG_L = 7;  const int ECHO_L = 8;
const int TRIG_R = 11; const int ECHO_R = 12;

// --- TUNING ---
#define BASE_SPEED  60
#define WALL_KP     2     // Sensitivity

// --- STRICT RANGES ---
#define MIN_WALL_DIST 6
```

```
#define MAX_WALL_DIST 20

void setup() {
  // Motor Setup
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);
  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);

  // Sensor Setup
  pinMode(TRIG_L, OUTPUT); pinMode(ECHO_L, INPUT);
  pinMode(TRIG_R, OUTPUT); pinMode(ECHO_R, INPUT);

  Serial.begin(9600);
  Serial.println("--- STRICT TUNNEL MODE START ---");
  delay(2000);
}

void loop() {
  // 1. READ SENSORS
  int distLeft  = getDistance(TRIG_L, ECHO_L);
  int distRight = getDistance(TRIG_R, ECHO_R);

  Serial.print("L: "); Serial.print(distLeft);
  Serial.print(" | R: "); Serial.println(distRight);

  // 2. CHECK VALIDITY
  bool validLeft  = (distLeft >= MIN_WALL_DIST && distLeft <= MAX_WALL_DIST);
```

```
  bool validRight = (distRight >= MIN_WALL_DIST && distRight <= MAX_WALL_DIST);

  // 3. STRICT DECISION (AND Logic)
  if (validLeft && validRight) {
    // CONDITION: BOTH sensors are safe.
    // ACTION: Calculate error and move.

    int error = distLeft - distRight;
    int correction = error * WALL_KP;

    move(BASE_SPEED - correction, BASE_SPEED + correction);
  }
  else {
    // CONDITION: One or Both sensors are unsafe.
    // ACTION: Immediate Stop.
    stopMotors();
  }

  delay(50); // Loop stability
}

// --- HELPERS ---

int getDistance(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
```

```
  digitalWrite(trig, LOW);

  long duration = pulseIn(echo, HIGH, 30000);
  if (duration == 0) return 999;
  return duration * 0.0343 / 2;
}

void move(int leftSpeed, int rightSpeed) {
  leftSpeed = constrain(leftSpeed, 0, 120);
  rightSpeed = constrain(rightSpeed, 0, 120);

  if (leftSpeed > 0) {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, leftSpeed);
  } else {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, 0);
  }

  if (rightSpeed > 0) {
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, rightSpeed);
  } else {
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, 0);
  }
}

void stopMotors() {
  analogWrite(L_PWM, 0); analogWrite(R_PWM, 0);
```

```
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}
```

Color sensor code

```
#include <Wire.h>
#include "Adafruit_TCS34725.h"

Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
TCS34725_GAIN_1X);

void setup() {
  Serial.begin(9600);
  if (!tcs.begin()) {
    Serial.println("No TCS34725 found ... check your connections");
    while (1);
  }
}

void loop() {
  float r, g, b;

  // This function gets the RGB data tailored for standard RGB (0-255)
  tcs.getRGB(&r, &g, &b);

  Serial.print("R: "); Serial.print((int)r); Serial.print(" ");
  Serial.print("G: "); Serial.print((int)g); Serial.print(" ");
```

```
  Serial.print("B: "); Serial.print((int)b); Serial.print(" ");


  // Simple logic test
  if ((int)r > (int)g && (int)r > (int)b) {
    Serial.println("-> Detected: RED");
  }
  else if ((int)g > (int)r && (int)g > (int)b) {
    Serial.println("-> Detected: GREEN");
  }
  else if ((int)b > (int)r && (int)b > (int)g) {
    Serial.println("-> Detected: BLUE");
  }
  else {
    Serial.println("-> Ambiguous");
  }


  delay(100);
}
```

4.Merged final (Line following + tunnel)

```
/*
 * ChromoBot - MERGED FINAL
 * ----------------------------------------------
 * PHASE 1: White Line Following (with Bend/Drift Logic)
 * PHASE 2: Strict Tunnel Navigation (Starts when Line is lost/Black Floor detected)
 * * PINS:
 * - Motors: Left(2, 22, 23) | Right(3, 24, 25)
 * - IR Sensors: 44, 45, 46, 47, 48
 * - Ultrasonic: Left(7,8) | Right(11,12)
 */

// ==========================================
//            PIN DEFINITIONS
// ==========================================

// --- MOTORS ---
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;

// --- IR SENSORS (Line) ---
const int S1 = 44; const int S2 = 45; const int S3 = 46; const int S4 = 47; const int S5 = 48;

// --- ULTRASONIC SENSORS (Tunnel) ---
const int TRIG_L = 7;  const int ECHO_L = 8;
const int TRIG_R = 11; const int ECHO_R = 12;
```

```
// ==========================================
//            TUNING VARS
// ==========================================

// --- LINE FOLLOWER TUNING ---
#define PID_SPEED   60
#define ALIGN_SPEED 60
#define TURN_SPEED  100
#define DRIFT_LIMIT 400     // Max time to rotate for drift (ms)
#define ALIGN_TIME  1200    // 1.2 Seconds Blind Forward

#define KP 18.0
#define KD 22.0

// --- TUNNEL TUNING ---
#define TUNNEL_BASE_SPEED  60
#define WALL_KP           2      // Sensitivity
#define MIN_WALL_DIST      6
#define MAX_WALL_DIST      20

// --- LOGIC MAP ---
#define LINE 1   // White
#define FLOOR 0  // Black

// --- GLOBALS ---
```

```
int lastError = 0;
bool isRunning = true; // Main Line Follower Flag

void setup() {
  // Motor Pins
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);
  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);

  // IR Pins
  pinMode(S1, INPUT); pinMode(S2, INPUT); pinMode(S3, INPUT); pinMode(S4, INPUT);
pinMode(S5, INPUT);

  // Ultrasonic Pins
  pinMode(TRIG_L, OUTPUT); pinMode(ECHO_L, INPUT);
  pinMode(TRIG_R, OUTPUT); pinMode(ECHO_R, INPUT);

  Serial.begin(9600);
  Serial.println("--- READY: LINE MODE -> TUNNEL MODE ---");
  delay(2000);
}

// ==========================================
//             MAIN LOOP
// ==========================================
void loop() {
  if (!isRunning) return; // If we finished or stopped, do nothing.
```

```
int s1 = digitalRead(S1);
int s2 = digitalRead(S2);
int s3 = digitalRead(S3);
int s4 = digitalRead(S4);
int s5 = digitalRead(S5);
int activeSensors = s1 + s2 + s3 + s4 + s5;

// ----------------------------------------
// TRANSITION: CHECK FOR LINE LOSS (All Black)
// ----------------------------------------
if (activeSensors == 0) {
  delay(100); // 100ms Debounce
  if (digitalRead(S3) == FLOOR) {

    Serial.println("--- LINE LOST: SWITCHING TO TUNNEL ---");
    stopMotors();
    delay(1000);      // Brief pause before tunnel logic starts
    navigateTunnel();  // <--- JUMPS TO TUNNEL LOGIC FOREVER

    isRunning = false; // Effectively ends the main loop
    return;
  }
}

// ----------------------------------------
```

```
// PHASE 1: LINE FOLLOWING LOGIC
// ----------------------------------------

// PRIORITY 1: BEND DETECTION
if (activeSensors >= 4) {
  if (s1 == LINE && s2 == LINE && s3 == LINE && s4 == LINE) {
    handleBend(-1); return;
  }
  if (s2 == LINE && s3 == LINE && s4 == LINE && s5 == LINE) {
    handleBend(1); return;
  }
}

// PRIORITY 2: DRIFT DETECTION
if (activeSensors == 3) {
  if (s1 == LINE && s2 == LINE && s3 == LINE) {
    handleDrift(-1); return;
  }
  if (s3 == LINE && s4 == LINE && s5 == LINE) {
    handleDrift(1); return;
  }
}

// PRIORITY 3: NORMAL PID
if (activeSensors > 0) {
  int error = (s1 * -4) + (s2 * -2) + (s3 * 0) + (s4 * 2) + (s5 * 4);
```

```
    error = error / activeSensors;

    int P = error;
    int D = error - lastError;
    int correction = (KP * P) + (KD * D);
    lastError = error;

    move(PID_SPEED + correction, PID_SPEED - correction);
  }
}

// ==========================================
//            PHASE 2: TUNNEL LOGIC
// ==========================================

void navigateTunnel() {
  Serial.println("--- TUNNEL MODE ACTIVE ---");

  // Locked Loop: Robot stays here forever (or until restart)
  while (true) {

    // 1. READ SENSORS
    int distLeft  = getDistance(TRIG_L, ECHO_L);
    int distRight = getDistance(TRIG_R, ECHO_R);

    // 2. CHECK VALIDITY (Strict 10-20cm logic)
```

```
bool validLeft  = (distLeft >= MIN_WALL_DIST && distLeft <= MAX_WALL_DIST);
bool validRight = (distRight >= MIN_WALL_DIST && distRight <= MAX_WALL_DIST);

// 3. STRICT DECISION (AND Logic)
if (validLeft && validRight) {
  // BOTH SAFE -> MOVE
  int error = distLeft - distRight;
  int correction = error * WALL_KP;

  int leftSpeed = TUNNEL_BASE_SPEED - correction;
  int rightSpeed = TUNNEL_BASE_SPEED + correction;

  // CLAMP: Mimic the strict tunnel code (Don't reverse, just stop wheel)
  if (leftSpeed < 0) leftSpeed = 0;
  if (rightSpeed < 0) rightSpeed = 0;
  // Clamp Max
  if (leftSpeed > 120) leftSpeed = 120;
  if (rightSpeed > 120) rightSpeed = 120;

  move(leftSpeed, rightSpeed);

} else {
  // ONE OR BOTH UNSAFE -> STOP
  stopMotors();
  // Robot waits here until walls are valid again (e.g., if you manually reset it)
  // or it just stays stopped as requested.
```

```
    }

    delay(50); // Loop stability
  }
}

// ==========================================
//              HELPERS
// ==========================================

int getDistance(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);

  long duration = pulseIn(echo, HIGH, 30000);
  if (duration == 0) return 999;
  return duration * 0.0343 / 2;
}

void handleBend(int direction) {
  stopMotors(); delay(1000);
  move(ALIGN_SPEED, ALIGN_SPEED); delay(ALIGN_TIME);
  stopMotors(); delay(1000);
  spinUntilMiddle3(direction, 6000);
}
```

```
void handleDrift(int direction) {
  stopMotors(); delay(1000);
  spinUntilMiddle3(direction, DRIFT_LIMIT);
}

void spinUntilMiddle3(int direction, int timeoutMs) {
  unsigned long startTime = millis();

  if (direction == -1) {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, TURN_SPEED);
    digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, TURN_SPEED);
  } else {
    digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, TURN_SPEED);
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, TURN_SPEED);
  }

  delay(100);

  while (millis() - startTime < timeoutMs) {
    int s2 = digitalRead(S2);
    int s3 = digitalRead(S3);
    int s4 = digitalRead(S4);

    if (s2 == LINE && s3 == LINE && s4 == LINE) {
      stopMotors(); delay(200); return;
```

```
    }
  }
  stopMotors();
}

// --- GLOBAL MOTOR DRIVER ---
void move(int leftSpeed, int rightSpeed) {
  // Constrain generally to PWM limits
  leftSpeed = constrain(leftSpeed, -255, 255);
  rightSpeed = constrain(rightSpeed, -255, 255);

  if (leftSpeed >= 0) {
    digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); analogWrite(L_PWM, leftSpeed);
  } else {
    digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); analogWrite(L_PWM, abs(leftSpeed));
  }

  if (rightSpeed >= 0) {
    digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); analogWrite(R_PWM, rightSpeed);
  } else {
    digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); analogWrite(R_PWM, abs(rightSpeed));
  }
}

void stopMotors() {
  analogWrite(L_PWM, 0); analogWrite(R_PWM, 0);
```

```
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}
```

5.Final (without color sorting)

```
/*
  ChromoBot - STRICT LOGIC (CORRECTED)
  Deterministic state machine implementation
*/

enum RobotState {
 LINE_FOLLOW,
 TUNNEL,
 POST_TUNNEL_ALIGN,
 SEQUENCE,
 FINAL_STOP
};

RobotState state = LINE_FOLLOW;

// ================= PINS =================
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;

const int S1 = 44; const int S2 = 45; const int S3 = 46; const int S4 = 47; const int S5 = 48;

const int TRIG_L = 7; const int ECHO_L = 8;
const int TRIG_R = 11; const int ECHO_R = 12;
const int TRIG_MID = 10; const int ECHO_MID = 9;
```

```
// ================= TUNING =================
#define PID_SPEED 60
#define TURN_SPEED 100
#define ALIGN_SPEED 60
#define ALIGN_TIME 350

#define KP 18.0
#define KD 22.0

#define TUNNEL_BASE_SPEED 60
#define WALL_KP 3

#define PICKUP_MIN_DIST 10
#define PICKUP_MAX_DIST 20

#define LINE 1
#define FLOOR 0

// ================= GLOBALS =================
int lastError = 0;
int tJunctionsPassed = 0;
bool tunnelFinished = false;

// ==========================================

void setup() {
```

```
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);

  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);

  pinMode(S1, INPUT); pinMode(S2, INPUT); pinMode(S3, INPUT); pinMode(S4, INPUT);
pinMode(S5, INPUT);

  pinMode(TRIG_L, OUTPUT); pinMode(ECHO_L, INPUT);

  pinMode(TRIG_R, OUTPUT); pinMode(ECHO_R, INPUT);

  pinMode(TRIG_MID, OUTPUT); pinMode(ECHO_MID, INPUT);

}


void loop() {


  if (state == FINAL_STOP) return;


  int s1 = digitalRead(S1);

  int s2 = digitalRead(S2);

  int s3 = digitalRead(S3);

  int s4 = digitalRead(S4);

  int s5 = digitalRead(S5);

  int active = s1 + s2 + s3 + s4 + s5;


  // ================= FINAL PICKUP =================

  if (state == SEQUENCE && tJunctionsPassed == 2) {

    int d = getDistance(TRIG_MID, ECHO_MID);

    if (d >= PICKUP_MIN_DIST && d <= PICKUP_MAX_DIST) {

      stopMotors();

      state = FINAL_STOP;
```

```
    return;
  }
}

// ================= LINE LOSS → TUNNEL =================
if (state == LINE_FOLLOW && active == 0) {
  delay(50);
  if (digitalRead(S3) == FLOOR) {
    stopMotors();
    delay(300);
    state = TUNNEL;
    return;
  }
}

// ================= TUNNEL MODE =================
if (state == TUNNEL) {
  if (s1 == LINE || s3 == LINE || s5 == LINE) {
    stopMotors();
    state = POST_TUNNEL_ALIGN;
    return;
  }

  int dl = getDistance(TRIG_L, ECHO_L);
  int dr = getDistance(TRIG_R, ECHO_R);
  int error = dl - dr;
```

```
  int corr = error * WALL_KP;
  move(TUNNEL_BASE_SPEED - corr, TUNNEL_BASE_SPEED + corr);
  delay(40);
  return;
}

// ================= POST TUNNEL ALIGN =================
if (state == POST_TUNNEL_ALIGN) {

  if (s2 && s3 && s4) {
    tunnelFinished = true;
    lastError = 0;
    state = SEQUENCE;
    return;
  }

  if (s1 || s2) spinUntilMiddle3(-1);
  else if (s4 || s5) spinUntilMiddle3(1);

  if (digitalRead(S2) && digitalRead(S3) && digitalRead(S4)) {
    tunnelFinished = true;
    lastError = 0;
    state = SEQUENCE;
  }

  return;
```

```
}

// ================= INTERSECTION DETECTION =================
if ((active >= 4) ||
   (s1 && s2) ||
   (s4 && s5)) {

  move(PID_SPEED, PID_SPEED);
  delay(50);
  stopMotors();

  int r1 = digitalRead(S1);
  int r2 = digitalRead(S2);
  int r3 = digitalRead(S3);
  int r4 = digitalRead(S4);
  int r5 = digitalRead(S5);
  int rActive = r1 + r2 + r3 + r4 + r5;

  bool confirmed =
    (rActive >= 4) ||
    (r1 && r2) ||
    (r4 && r5);

  if (confirmed) {

    if (state == SEQUENCE) {
```

```
      if (tJunctionsPassed < 2) {
        executeLeftTurn();
        tJunctionsPassed++;
      }

    } else {
      if (r1 && r2) executeLeftTurn();
      else if (r4 && r5) executeRightTurn();
    }

    return;
  }
}

// ================= PID LINE FOLLOW =================
if (active > 0) {

  int error = (s1 * -4) + (s2 * -2) + (s3 * 0) + (s4 * 2) + (s5 * 4);
  error /= active;

  int P = error;
  int D = error - lastError;
  int corr = (KP * P) + (KD * D);
  lastError = error;
```

```
    move(PID_SPEED + corr, PID_SPEED - corr);
  }
}

// ==========================================
// TURN SEQUENCES
// ==========================================

void executeLeftTurn() {
  stopMotors();
  delay(200);
  move(ALIGN_SPEED, ALIGN_SPEED);
  delay(ALIGN_TIME);
  stopMotors();
  spinUntilMiddle3(-1);
}

void executeRightTurn() {
  stopMotors();
  delay(200);
  move(ALIGN_SPEED, ALIGN_SPEED);
  delay(ALIGN_TIME);
  stopMotors();
  spinUntilMiddle3(1);
}
```

```
void spinUntilMiddle3(int dir) {

  if (dir == -1)
    move(-TURN_SPEED, TURN_SPEED);
  else
    move(TURN_SPEED, -TURN_SPEED);

  while (true) {
    if (digitalRead(S2) && digitalRead(S3) && digitalRead(S4)) {
      stopMotors();
      delay(150);
      return;
    }
  }
}

// ==========================================

int getDistance(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long dur = pulseIn(echo, HIGH, 30000);
  if (dur == 0) return 999;
  return dur * 0.0343 / 2;
}
```

```
void move(int l, int r) {
  l = constrain(l, -255, 255);
  r = constrain(r, -255, 255);

  if (l >= 0) { digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); }
  else { digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); }
  analogWrite(L_PWM, abs(l));

  if (r >= 0) { digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); }
  else { digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); }
  analogWrite(R_PWM, abs(r));
}

void stopMotors() {
  analogWrite(L_PWM, 0);
  analogWrite(R_PWM, 0);
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}
```

6. Final code

```
#include <Wire.h>

#include "Adafruit_TCS34725.h"


// ================= SETUP TCS34725 =================

// 50ms integration time, 1x gain for accurate close-range reading

Adafruit_TCS34725 tcs = Adafruit_TCS34725(TCS34725_INTEGRATIONTIME_50MS,
TCS34725_GAIN_1X);


// ================= STATES =================

enum RobotState {

  LINE_FOLLOW,         // Phase 1: Start -> Tunnel

  TUNNEL,              // Phase 2: Wall Follow

  POST_TUNNEL_ALIGN,    // Phase 3: Exit Alignment

  SEQUENCE_BLUE,       // Phase 3: Navigate to Blue Tower

  COLOR_CHECK_BLUE,     // Phase 4: Identify Blue

  TO_DROP_BLUE,        // Phase 5: Go to Drop Point (Ignore Right)

  DROP_ACTION_BLUE,     // Phase 5: Drop Logic

  TO_GREEN_TOWER,      // Phase 6: Navigate to Green (Left -> Ignore Right -> Right)

  COLOR_CHECK_GREEN,    // Phase 6: Identify Green

  TO_DROP_GREEN,       // Phase 7: Go to Drop Point (Ignore Left)

  DROP_ACTION_GREEN,    // Phase 7: Drop Logic

  MISSION_COMPLETE      // Stop

};


RobotState state = LINE_FOLLOW;
```

```
// ================= PINS =================
// MOTORS
const int L_PWM = 2; const int L_IN1 = 22; const int L_IN2 = 23;
const int R_PWM = 3; const int R_IN1 = 24; const int R_IN2 = 25;

// IR SENSORS (Left to Right)
const int S1 = 44; const int S2 = 45; const int S3 = 46; const int S4 = 47; const int S5 = 48;

// ULTRASONICS
const int TRIG_L = 7;   const int ECHO_L = 8;
const int TRIG_R = 11;  const int ECHO_R = 12;
const int TRIG_MID = 10; const int ECHO_MID = 9;

// ================= TUNING =================
#define PID_SPEED   60
#define TURN_SPEED  100
#define ALIGN_SPEED 60

// TIMERS (Adjust these based on robot speed/battery)
#define ALIGN_TIME  350     // Time to drive STRAIGHT before turning
#define BLIND_TIME  400     // Time to ignore junctions (Loophole Fix)
#define UTURN_TIME  800     // Blind spin time for U-Turn

// PID
#define KP 18.0
```

```
#define KD 22.0

// TUNNEL
#define TUNNEL_BASE_SPEED 60
#define WALL_KP 3

// DISTANCES
#define PICKUP_MIN 8
#define PICKUP_MAX 20

// LOGIC
#define LINE 1
#define FLOOR 0

// ================= GLOBALS =================
int lastError = 0;
int tJunctionsPassed = 0; // Re-used counter for phases
bool tunnelFinished = false;

// ==========================================

void setup() {
  Serial.begin(9600);

  // Motor Pins
  pinMode(L_PWM, OUTPUT); pinMode(L_IN1, OUTPUT); pinMode(L_IN2, OUTPUT);
```

```
  pinMode(R_PWM, OUTPUT); pinMode(R_IN1, OUTPUT); pinMode(R_IN2, OUTPUT);

  // Sensor Pins
  pinMode(S1, INPUT); pinMode(S2, INPUT); pinMode(S3, INPUT); pinMode(S4, INPUT);
pinMode(S5, INPUT);
  pinMode(TRIG_L, OUTPUT); pinMode(ECHO_L, INPUT);
  pinMode(TRIG_R, OUTPUT); pinMode(ECHO_R, INPUT);
  pinMode(TRIG_MID, OUTPUT); pinMode(ECHO_MID, INPUT);

  // Color Sensor Init
  if (tcs.begin()) {
    Serial.println("Sensor Found");
  } else {
    Serial.println("No TCS34725 found ... check connections");
    while (1); // Halt
  }

  Serial.println("--- MISSION START ---");
  delay(2000);
}

void loop() {
  if (state == MISSION_COMPLETE) {
      stopMotors();
      return;
  }
```

```
// READ SENSORS
int s1 = digitalRead(S1); int s2 = digitalRead(S2);
int s3 = digitalRead(S3); int s4 = digitalRead(S4); int s5 = digitalRead(S5);
int active = s1 + s2 + s3 + s4 + s5;

// ============================================
//            PHASE 7: GREEN DROP
// ============================================
if (state == TO_DROP_GREEN) {
  // 1. ARRIVAL: Line Ends (All Black)
  if (active == 0) {
    // Double check to ensure it's not a gap
    delay(50);
    if (digitalRead(S3) == FLOOR) {
      // Logic says active 0 is black line, but confirm it's not a hole
      // Assuming White Line on Black Floor: 0 active means NO LINE (Black).
      stopMotors();
      state = DROP_ACTION_GREEN;
      return;
    }
  }

  // 2. LOOPHOLE FIX: Ignore LEFT Turns (Passing 3rd T-Junc)
  // If we see Left Wing (S1+S2) or Junction (4+)
  if (active >= 4 || (s1 && s2)) {
```

```
        Serial.println("-> Ignoring Left Turn (Blind Forward)");
        ignoreTurn();
        return;
    }

    runPID(s1, s2, s3, s4, s5, active);
    return;
}

if (state == DROP_ACTION_GREEN) {
    Serial.println("--- DROPPING GREEN CUBE ---");
    delay(1000); // Simulate Drop
    state = MISSION_COMPLETE;
    return;
}

// ==========================================
//            PHASE 6: TO GREEN TOWER
// ==========================================
if (state == TO_GREEN_TOWER) {

    // Check Ultrasonic for Arrival (Only valid after the final Right turn)
    if (tJunctionsPassed == 3) {
        int d = getDistance(TRIG_MID, ECHO_MID);
        if (d >= PICKUP_MIN && d <= PICKUP_MAX) {
            stopMotors();
```

```
        state = COLOR_CHECK_GREEN;
        return;
    }
}

// Intersection Handling
if (active >= 4 || (s1&&s2) || (s4&&s5)) {
    if (verifyIntersection()) {

        // Turn 1: 1st T-Junc -> LEFT
        if (tJunctionsPassed == 0) {
            executeTurn(-1); // Left
            tJunctionsPassed++;
            return;
        }

        // Loophole: Passing Tunnel T (Right Side) -> IGNORE
        if (tJunctionsPassed == 1) {
             Serial.println("-> Crossing Tunnel T (Blind Forward)");
             ignoreTurn();
             tJunctionsPassed++; // Increment to signify we passed it
             return;
        }

        // Turn 2: 3rd T-Junc -> RIGHT
        if (tJunctionsPassed == 2) {
```

```
            executeTurn(1); // Right
            tJunctionsPassed++;
            return;
        }
    }
  }
  runPID(s1, s2, s3, s4, s5, active);
  return;
}

if (state == COLOR_CHECK_GREEN) {
  checkColorAverage(false); // Check Green
  performUTurn();
  state = TO_DROP_GREEN;
  return;
}

// ==========================================
//          PHASE 5: TO BLUE DROP
// ==========================================
if (state == TO_DROP_BLUE) {
  // 1. ARRIVAL: Line Ends
  if (active == 0) {
    delay(50);
    // Assuming active 0 = All Black (Drop Point)
    stopMotors();
```

```
        state = DROP_ACTION_BLUE;
        return;
    }

    // 2. LOOPHOLE FIX: Ignore RIGHT Turns (Passing 1st T-Junc)
    if (active >= 4 || (s4 && s5)) {
        Serial.println("-> Ignoring Right Turn (Blind Forward)");
        ignoreTurn();
        return;
    }

    runPID(s1, s2, s3, s4, s5, active);
    return;
}

if (state == DROP_ACTION_BLUE) {
    Serial.println("--- DROPPING BLUE CUBE ---");
    delay(1000);

    performUTurn();
    tJunctionsPassed = 0; // Reset counter for Phase 6
    state = TO_GREEN_TOWER;
    return;
}

// ==========================================
```

```
//          PHASE 4: COLOR CHECK 1
// ==========================================
if (state == COLOR_CHECK_BLUE) {
   checkColorAverage(true); // Check Blue

   performUTurn();
   state = TO_DROP_BLUE;
   return;
}

// ==========================================
//          PHASE 3: SEQUENCE BLUE
// ==========================================
if (state == SEQUENCE_BLUE) {
   // Check Arrival
   if (tJunctionsPassed >= 2) {
      int d = getDistance(TRIG_MID, ECHO_MID);
      if (d >= PICKUP_MIN && d <= PICKUP_MAX) {
         stopMotors();
         state = COLOR_CHECK_BLUE;
         return;
      }
   }

   // Check Turns
   if (active >= 4 || (s1&&s2) || (s4&&s5)) {
```

```
        if (verifyIntersection()) {
            if (tJunctionsPassed < 2) {
                executeTurn(-1); // Force Left x2
                tJunctionsPassed++;
            }
        }
    }
    runPID(s1, s2, s3, s4, s5, active);
    return;
}

// ==========================================
//            PHASE 2: TUNNEL EXIT
// ==========================================
if (state == POST_TUNNEL_ALIGN) {
    // Auto-Balance: Scan if we are crooked
    if (s2 && s3 && s4) {
        stopMotors(); delay(200);
        state = SEQUENCE_BLUE;
        tJunctionsPassed = 0;
        return;
    }
    if (s1 || s2) spinUntilMiddle3(-1); // Spin Left
    else if (s4 || s5) spinUntilMiddle3(1); // Spin Right

    // Failsafe: if s3 only is active, we are good
```

```
    if (s3) {
        state = SEQUENCE_BLUE;
        tJunctionsPassed = 0;
    }
    return;
}

if (state == TUNNEL) {
    // Exit Condition: Any Line Detected
    if (s1 == LINE || s3 == LINE || s5 == LINE) {
        stopMotors();
        delay(500); // Settle
        state = POST_TUNNEL_ALIGN;
        return;
    }
    // Wall Follow
    int dl = getDistance(TRIG_L, ECHO_L);
    int dr = getDistance(TRIG_R, ECHO_R);
    int error = dl - dr;
    int corr = error * WALL_KP;
    move(TUNNEL_BASE_SPEED - corr, TUNNEL_BASE_SPEED + corr);
    delay(40);
    return;
}

// ==========================================
```

```
  //          PHASE 1: LINE FOLLOW
  // ==========================================
  if (state == LINE_FOLLOW) {
    // Check Tunnel Entry
    if (active == 0) {
      delay(50);
      if (digitalRead(S3) == FLOOR) { // Confirm All Black + Floor Ground
        stopMotors(); delay(500);
        state = TUNNEL;
        return;
      }
    }

    // Phase 1 Turns (Standard)
    if (active >= 4 || (s1&&s2) || (s4&&s5)) {
      if (verifyIntersection()) {
        if (s1 && s2) executeTurn(-1);     // Left
        else if (s4 && s5) executeTurn(1);  // Right
      }
    }
    runPID(s1, s2, s3, s4, s5, active);
  }
}

// ==========================================
//          HELPER FUNCTIONS
```

```
// ==========================================

// ROBUST COLOR CHECKER
void checkColorAverage(bool lookForBlue) {
  long rSum = 0, gSum = 0, bSum = 0;

  // Take 5 samples
  for(int i=0; i<5; i++) {
    float r, g, b;
    tcs.getRGB(&r, &g, &b);
    rSum += (long)r; gSum += (long)g; bSum += (long)b;
    delay(20);
  }
  long r = rSum/5; long g = gSum/5; long b = bSum/5;

  Serial.print("Avg Color: R"); Serial.print(r); Serial.print(" G"); Serial.print(g); Serial.print(" B");
Serial.println(b);

  // Simple Ratio Logic (Adjust 1.2 multiplier if needed)
  bool isBlue  = (b > r * 1.2 && b > g * 1.2);
  bool isGreen = (g > r * 1.2 && g > b * 1.2);

  if (lookForBlue && isBlue) {
    Serial.println("-> BLUE CONFIRMED: Picking Up...");
    delay(2000); // Simulate Arm
  }
```

```
    else if (!lookForBlue && isGreen) {
        Serial.println("-> GREEN CONFIRMED: Picking Up...");
        delay(2000); // Simulate Arm
    }
    else {
        Serial.println("-> Color Mismatch or Ambiguous");
        delay(500);
    }
}

void performUTurn() {
    Serial.println("-> U-TURN");
    stopMotors(); delay(500);
    move(TURN_SPEED, -TURN_SPEED); // Spin
    delay(UTURN_TIME); // Blind Spin to clear line
    spinUntilMiddle3(1); // Seek Line (Spin Right)
}

void ignoreTurn() {
    // Blindly move forward to clear the intersection
    move(PID_SPEED, PID_SPEED);
    delay(BLIND_TIME);
}

bool verifyIntersection() {
    move(PID_SPEED, PID_SPEED);
```

```
    delay(50); // Move into the intersection slightly
    stopMotors();

    int r1 = digitalRead(S1); int r2 = digitalRead(S2);
    int r3 = digitalRead(S3); int r4 = digitalRead(S4); int r5 = digitalRead(S5);
    int active = r1 + r2 + r3 + r4 + r5;

    return (active >= 3);
}

void executeTurn(int dir) {
    stopMotors(); delay(200);
    move(ALIGN_SPEED, ALIGN_SPEED);
    delay(ALIGN_TIME); // Align wheels with center of turn
    stopMotors();
    spinUntilMiddle3(dir);
}

void spinUntilMiddle3(int dir) {
    if (dir == -1) move(-TURN_SPEED, TURN_SPEED); // Left
    else move(TURN_SPEED, -TURN_SPEED); // Right

    // Safety timeout to prevent infinite loop
    unsigned long start = millis();
    while (millis() - start < 5000) {
        if (digitalRead(S2) && digitalRead(S3) && digitalRead(S4)) {
```

```
        stopMotors(); delay(150); return;
      }
    }
    stopMotors(); // Timeout stop
}

void runPID(int s1, int s2, int s3, int s4, int s5, int active) {
    if (active == 0) return;
    int error = (s1 * -4) + (s2 * -2) + (s3 * 0) + (s4 * 2) + (s5 * 4);
    error /= active;

    int P = error;
    int D = error - lastError;
    int corr = (KP * P) + (KD * D);
    lastError = error;

    move(PID_SPEED + corr, PID_SPEED - corr);
}

void move(int l, int r) {
    l = constrain(l, -255, 255); r = constrain(r, -255, 255);
    if (l >= 0) { digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, HIGH); }
    else { digitalWrite(L_IN1, HIGH); digitalWrite(L_IN2, LOW); }
    analogWrite(L_PWM, abs(l));

    if (r >= 0) { digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, HIGH); }
```

```
  else { digitalWrite(R_IN1, HIGH); digitalWrite(R_IN2, LOW); }
  analogWrite(R_PWM, abs(r));
}

void stopMotors() {
  analogWrite(L_PWM, 0); analogWrite(R_PWM, 0);
  digitalWrite(L_IN1, LOW); digitalWrite(L_IN2, LOW);
  digitalWrite(R_IN1, LOW); digitalWrite(R_IN2, LOW);
}

int getDistance(int trig, int echo) {
  digitalWrite(trig, LOW); delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long dur = pulseIn(echo, HIGH, 30000);
  if (dur == 0) return 999;
  return dur * 0.0343 / 2;
}
```